

```
// Program for validating strings of  $a^nb^n \mid n > 0$ 
```

```
//p1.l
```

```
%%
```

```
a {return(A);}
```

```
b {return(B);}
```

```
. {return(yytext[0]);}
```

```
\n {return('\n');}
```

```
%%
```

```
int yywrap(){return 1;}
```

```
//p1.y
```

```
%token A B
```

```
%%
```

```
start: S '\n'      {printf("\n valid Statement\n");}
```

```
S: A B
```

```
| A S B
```

```
;
```

```
%%
```

```
#include"lex.yy.c"
```

```
int yyerror(char *s){
```

```
printf("\n%s",s); }
```

```
int main(){
```

```
yyparse();
```

```
}
```

```
=====
```

```
//Program #2 - validating the structure of an assignment statement
```

```
//assmt.l
```

```
%%
```

```
[a-zA-Z]([a-zA-Z0-9])* {return(ID);}
```

```
[0-9]+      {return(NUM);}
```

```
.          {return(yytext[0]);}
```

```
\n         {return('\n');}
```

```
%%
```

```
int yywrap(){
```

```
return 1;}
```

```
//assmt.y
```

```
%token ID,NUM
```

```
%left '+' '-'
```

```
%left '*' '/'
```

```
%%
```

```
start: S '\n'      {printf("\n valid Assignment Statement\n");}
```

```
S: ID '=' E
```

```
;
```

```
E: E '+' T
```

```
| E '-' T
```

```
| T
```

```
;
```

```
T: T '*' F
```

```
| T '/' F
```

```
| F
```

```
;
```

```
F: '(' E ')'
```

```
| ID
```

```

| NUM
;
%%
#include"lex.yy.c"
int yyerror(char *s){
printf("\n Error Message:%s",s);}

void main() {
yyparse(); }
=====
//Program #3 - validating the syntax of an expression
//exp.1
%%
[a-zA-Z] ([a-zA-Z0-9]) * {return(ID);}
.      {return(yytext[0]);}
\n     {return('\n');}
%%
int yywrap() {return 1;}

//exp.y

%token ID
%%
start: E '\n'      {printf("\n Valid\n");}
E: E '+' T
| T
;
T: ID
;
%%
#include"lex.yy.c"
int yyerror(char *s){
printf("\n Error: %s",s);}
void main(){
yyparse();}
=====
//program #4 - Program to validate the syntax of conditional statement
//cond.1
%{
#include<stdio.h>
%}
%%
if      {return(IF);}
\<     {return(LT);}
\>     {return(GT);}
==     {return(EQ);}
[a-zA-Z] ([a-zA-Z0-9]) * {return(ID);}
[0-9]+  {return(NUM);}
.      {return(yytext[0]);}
\n     {return('\n');}
%%
int yywrap() {return 1;}

```

//cond.y

```
%token IF LT GT EQ
%token ID NUM
%left '+' '-'
%%
start: S '\n'      {printf("Successful Parsing");return 0;}
S:IF '(' COND ')' '{ ' A '}'
  | A
;
COND: ID LT ID
     | ID GT ID
     | ID EQ ID
;
A: ID '=' E
;
E: E '+' T
  | T
;
T: ID
  | NUM
;
%%
#include"lex.yy.c"
int yyerror(char *s) {printf("\n%s", s);}
void main() {yyparse();}
```

=====

//Program #5 - validating the syntax of while statement

//whstat.1

```
{
#include<stdio.h>
}
%%
if      {return(IF);}
while {return(WHILE);}
[a-zA-Z]([a-zA-Z0-9])* {return(ID);}
\<      {return(LT);}
\>      {return(GT);}
==      {return(EQ);}
.        {return(yytext[0]);}
\n      {return('\n');}
%%
int yywrap() {return 1;}
```

//whstat.y

```
%token IF WHILE LT GT EQ
%token ID
%%
Start: S '\n'      {return 0;}
S:IS {printf("\n Valid If statement");}
  | WS {printf("\n valid While Loop");}
  | A {printf("\n Valid Assignment");}
;
```

```
IS: IF '(' COND ')' '{ ' A ' }'  
;  
WS: WHILE '(' COND ')' '{ ' IS ' }'  
;  
COND: ID R ID  
;  
R: LT  
| GT  
| EQ  
;  
A: ID '=' E  
;  
E: E '+' T  
| T  
;  
T: ID  
;  
%%  
#include "lex.yy.c"  
int yyerror(char *s) {printf("\n %s",s);}   
int main() {yyparse();}
```